

LLVM IR

Kedar Sedai

Abstract—The LLVM code generation phase translates optimized MiniLang HIR into textual LLVM IR. The emitter maps temporaries, locals, arithmetic, control flow and function calls to standard LLVM instructions. The minilang_compile driver completed the pipeline from source to executable using Clang, demonstrating end-to-end compilation.

I. INTRODUCTION

The LLVM(Low Level Virtual Machine) is much more than a VM today. It is a open source compiler used by Clang(C/C++), many research and production compilers. We choose C++ partly because LLVM’s native API is C++.LLVM provides intermediate language between compiler front end and the machine. Hundreds of optimization passes on IR. LLVM IR is a typed, SSA-oriented assembly-like language. Our emitter writes textual IR(.ll) files

A. Example from factorial program

```
define i32 @factorial(i32 %n) {
entry:
  %n.addr = alloca i32
  store i32 %n, i32* %n.addr
  %t1 = load i32, i32* %n.addr
  %t2 = add i32 0, 1
  %t0_cmp = icmp sle i32 %t1, %t2
  %t0 = zext i1 %t0_cmp to i32
  br i1 %t0, label %then_0, label
    %endif_2
then_0:
  ret i32 1
endif_2:
  ...
  ret i32 %t4
}
```

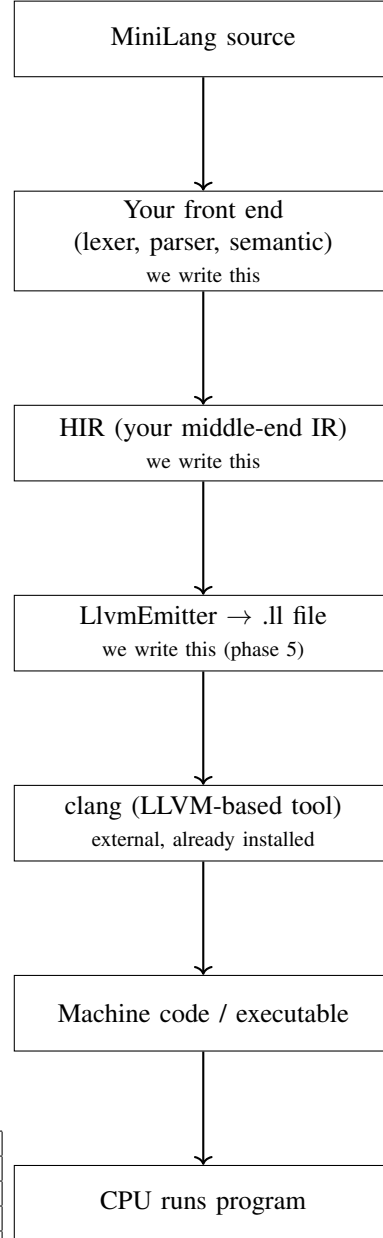
Reading this:

LLVM syntax	Meaning
define i32 @factorial(...)	Function returning 32-bit integer
%n.addr = alloca i32	Stack slot for local variable
load/store	Read/write memory
add, sub, mul	Integer arithmetic
icmp, sle	Signed compare ==
br i1..., label %A, label %B	Conditional branch
ret i32 %x	Return value
@main	Function symbol (like main)

TABLE I: Example of the factorial program

A HirModule holds HirFunction objects, each with a flat instructions vector.

II. HOW LLVM FITS INTO OUR COMPILER



Compiler at this point does not link against libLLVM yet. It prints LLVM IR as text. This is standard approach:

- Easier to debug (read .ll files)
- No LLVM SDK required to build MiniLang
- clang file.ll -o program compiles IR to and executable

Later we could use the LLVM C++ API to build IR in memory instead of printing text.

III. KEY LLVM CONCEPTS

A. SSA(Static Single Assignment)

Each value is assigned once. HIR temps(%t0, %t1) map naturally to LLVM values.

B. Basic blocks

Linear instruction sequences ending in br, or ret. HIR label/jump/br_cond become LLVM basic blocks.

C. Types

- i32 = 32-bit integer(int)
- i1 = 1-bit boolean
- void = no return value

D. IR vs machine code

IR is portable and optimizable; machine code is CPU-specific. LLVM translates IR -> assembly -> binary.

IV. WHY I CHOOSE LLVM INSTEAD OF WRITING ASSEMBLY YOURSELF?

Without LLVM	With LLVM
write x86, ARM, etc. separately	One IR, many targets
write your own register allocator	LLVM provides it
Limited optimizations	Mature optimizer(inlining, DCE, etc)
Huge backend effort	Focus on language design

TABLE II: Reason to choose LLVM

V. MAPPING HIR -> LLVM

HIR instruction	LLVM emitted
const_int	%tN = add i32 0, value
const_bool	%tN = add i32 0, 0/1
load_local	load from %name.addr
store_local	store to %name.addr
binary (+, -, *, /)	add, sub, mul, sdiv, srem
binary(<=, ==, ...)	icmp + zext to i32
binary(&&,)	icmp + and/or on i1
unary -	sub i32, 0, %op
call	call i32 @name(...)
ret	ret i32 %t or ret void
br_cond	br i1 %cond, label %then, label %else
jump	br label, %target
label	then_0: (basic block start)

TABLE III: Mapping HIR -> LLVM

VI. CONCLUSION

The MiniLang compiler now emits LLVM IR for native code generation. LLVM provides a portable, optimizable target so the project can focus on language design and future ML-guided optimizations rather than machine-specific backends.